14 Garras Auto-Publisher — Technical Documentation

For YouTube API Services Audit Date: 2026-05-17 **Operator:** Jonatas Pereira (`jonatas.freire.prof@gmail.com`) **Channel:** 14 Garras (`UCGXNSUQTScWqdN7Rfhkbg2A`) **Project:** `garras-496602` (Number `564689870047`) **Repository:** https://github.com/Jonatas-F/canal-cortes

---

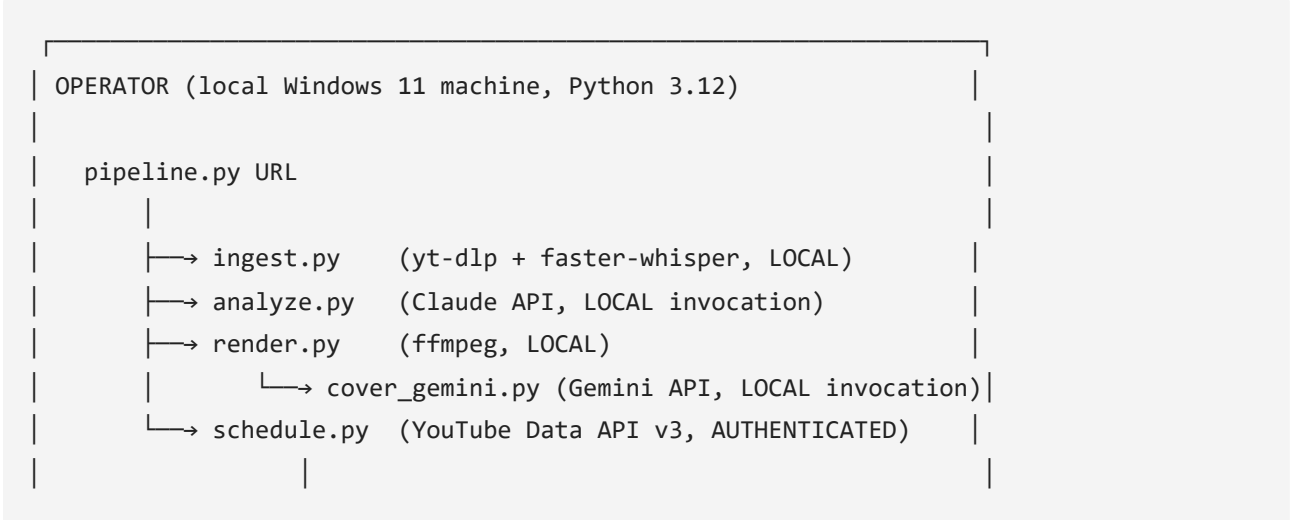
1. Overview

14 Garras Auto-Publisher is a single-user, command-line Python application that automates the publication of authorized podcast content cuts to a single YouTube channel. It runs locally on the operator's machine and is invoked manually for each batch of content.

The application has **no end users** other than the operator/channel owner. It does not expose any web interface, does not accept third-party authentication, and does not store data beyond a local SQLite queue tracking publication status.

---

2. Architecture



All YouTube API calls are made via the official `google-api-python-client` Python library (v2.196+).

---

3. OAuth 2.0 Authentication Flow

3.1 One-time setup

File: `scripts/auth_youtube.py`

```
 from google_auth_oauthlib.flow import InstalledAppFlow

SCOPES = [
    "https://www.googleapis.com/auth/youtube.upload",
    "https://www.googleapis.com/auth/youtube.readonly",
]

flow = InstalledAppFlow.from_client_secrets_file("client_secret.json", SCOPES)
creds = flow.run_local_server(port=0)
# Stores refresh_token in token.json (local only, gitignored)
```

Flow: 1. Operator runs `python scripts/auth_youtube.py` 2. Default browser opens at `https://accounts.google.com/o/oauth2/auth?...` 3. Operator logs in with their own Google account (channel owner) 4. Operator grants consent for the requested scopes 5. Authorization code is exchanged for access + refresh token 6. `token.json` is saved locally (in `.gitignore`)

3.2 Subsequent runs

File: `scripts/schedule.py`

```
 from google.oauth2.credentials import Credentials
from google.auth.transport.requests import Request

creds = Credentials.from_authorized_user_file("token.json")
if creds.expired and creds.refresh_token:
    creds.refresh(Request())  # silent refresh
    token.json.write_text(creds.to_json())
```

No user interaction required after initial setup. Token can be revoked at any time at https://myaccount.google.com/permissions.

---

4. API Operations

4.1 `videos.insert` — main upload operation

File: `scripts/schedule.py`, function `upload_to_youtube()`

```
 body = {
    "snippet": {
        "title": cut["titulo"],
        "description": cut["descricao"],  # includes "Source: <url>" attribution
        "tags": cut["tags"] + cut["hashtags"],
        "categoryId": "22",  # People & Blogs
        "defaultLanguage": "pt",
        "defaultAudioLanguage": "pt",
    },
    "status": {
        "privacyStatus": "private",     # uploaded as private
        "publishAt": "2026-05-20T19:00:00-03:00",  # YouTube native scheduling
        "selfDeclaredMadeForKids": False,
    },
}
media = MediaFileUpload(file_path, chunksize=-1, resumable=True)
response = yt.videos().insert(part="snippet,status", body=body, media_body=media).execute
```

Key behaviors: - All uploads use `privacyStatus=private` + `publishAt` — YouTube handles publication transition natively, no external scheduling required. - Description includes attribution link to the source video. - No artificial inflation: no view, like, subscribe, or comment operations.

4.2 `thumbnails.set` — custom thumbnails for long-form cuts only

```
yt.thumbnails().set(videoId=yt_id, media_body=MediaFileUpload(jpg_path, mimetype="image/
```

Only called for long-form cuts (3–10 min); Shorts use YouTube's default frame selection.

4.3 `channels.list` — verification only (read-only)

Called once during OAuth setup to verify channel identity:

```
yt.channels().list(part="snippet", mine=True).execute()
```

4.4 `videos.list` — verification only (read-only, optional)

Used optionally after upload to confirm scheduled state:

```
yt.videos().list(part="status", id=yt_id).execute()
```

---

5. Quota Management

File: `scripts/common.py`

A local SQLite table `youtube_quota` tracks daily API usage:

```
CREATE TABLE youtube_quota (
    date TEXT PRIMARY KEY,          -- YYYY-MM-DD (Pacific Time)
    units_used INTEGER NOT NULL DEFAULT 0,
    uploads_count INTEGER NOT NULL DEFAULT 0,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);
```

Before each `videos.insert`, the application checks:

```
def quota_can_upload(conn, daily_limit=10000, extra_units=0) -> bool:
    used, _ = quota_get_used_today(conn)
    needed = 1600 + extra_units  # videos.insert cost
    return (used + needed + 500) <= daily_limit  # 500 safety margin
```

If the safety margin would be exceeded, the upload is deferred to the next day. This prevents accidental quota exhaustion.

---

6. Source Material Authorization

The application is operated only on source video content for which the operator holds explicit re-publication authorization. Each source has a manifest file `inbox/<video_id>.json`:

```
{
  "autorizado": true,
  "canal_fonte": "Os Sócios Podcast",
  "source_url": "https://www.youtube.com/watch?v=...",
  "tema": "..."
}
```

The application checks the `autorizado` flag before adding cuts to the upload queue. If `autorizado=false`, the cut is marked as `blocked` and never uploaded.

---

7. Content Generated

Each cut uploaded contains: - **Original audio**: extracted from the authorized source video, trimmed to a specific segment (20s–10min) - **Burned-in subtitles**: word-by-word karaoke style (from Whisper transcription) - **Title overlay**: brief 3-second overlay at the start - **End card**: 3-second branded "14 Garras" subscribe call-to-action - **Custom thumbnail** (long-form only): generated via Gemini API using extracted speaker face references

Every video includes attribution to the source in its description:

```
Acompanhe a análise sobre...

📺 Vídeo original: https://www.youtube.com/watch?v=...

#politica #flaviobolsonaro #renansantos
```

---

8. Compliance Summary

YouTube API Policy	Compliance
Does not inflate views/likes/subs/comments	✅ No such API calls used
Does not scrape data from other channels	✅ Only uploads to operator's own channel
Respects quota limits	✅ Local quota tracker enforces ceiling
Uses `publishAt` for scheduling (not simulating user activity)	✅ Native YouTube scheduling only
Operates on authorized content only	✅ Manifest `autorizado=true` enforced
Source attribution in description	✅ Mandatory, auto-included
Single channel scope	✅ OAuth fixed to one Google account
No public-facing service	✅ Local CLI only

---

9. Data Handling

Stored locally (operator's machine only):

- `client_secret.json` — OAuth client credentials (gitignored)
- `token.json` — OAuth refresh token (gitignored)
- `queue.db` — SQLite tracking cut metadata, schedule, YouTube video IDs (gitignored)
- `cuts/<source_id>/*.mp4` — rendered cuts (deleted after publication)
- `raw/<source_id>/source.mp4` — downloaded source (deleted after all cuts uploaded)

Never collected or stored:

- Data from any user other than the operator
- Data from any YouTube channel other than 14 Garras
- Analytics, comments, or interaction data

Third-party services used:

- **Google YouTube Data API v3** (this audit)
- **Google Gemini API** (for thumbnail generation, no user data sent)
- **Anthropic Claude API** (for transcript analysis, local invocation)

---

10. References

- **Source code** (full review): https://github.com/Jonatas-F/canal-cortes
- **Privacy Policy**: https://jonatas-f.github.io/canal-cortes/legal/privacy-policy
- **Terms of Service**: https://jonatas-f.github.io/canal-cortes/legal/terms-of-service
- **Workflow documentation**: https://github.com/Jonatas-F/canal-cortes/blob/main/docs/SETUP.md
- **Channel**: https://www.youtube.com/channel/UCGXNSUQTScWqdN7Rfhkbg2A
- **Contact**: jonatas.freire.prof@gmail.com

---

11. Recorded Demonstration

A screen recording demonstrating the full pipeline (OAuth setup → cut generation → upload → scheduled state in YouTube Studio) is available upon request. Contact `jonatas.freire.prof@gmail.com`.